

COURSE CODE/TITLE:	CPE 212 - Automating Server Management	SECTION:	CPE31S2
DATE PERFORMED:	08 - 13 - 26	DATE SUBMITTED:	08 - 13 - 26
NAME:	Santiago, David Owen A.	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 1	Familiarization for the new normal Lab													
1. PROCEDURE OUTPUT														
Managed Nodes:														
<table><tr><th>Hostname</th><th>Operating System</th><th>IP Address</th></tr><tr><td>server1</td><td>Ubuntu</td><td>192.168.56.114</td></tr><tr><td>server2</td><td>Ubuntu</td><td>192.168.56.115</td></tr><tr><td>centos</td><td>CentOS</td><td>192.168.56.118</td></tr></table>			Hostname	Operating System	IP Address	server1	Ubuntu	192.168.56.114	server2	Ubuntu	192.168.56.115	centos	CentOS	192.168.56.118
Hostname	Operating System	IP Address												
server1	Ubuntu	192.168.56.114												
server2	Ubuntu	192.168.56.115												
centos	CentOS	192.168.56.118												
Part A: Installing and Verifying Ansible														
Ansible version <pre>Processing triggers for install-info (6.8-4build1) ... workstation@workstation:~\$ ansible --version ansible [core 2.16.3] config file = None configured module search path = ['/home/workstation/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules'] ansible python module location = /usr/lib/python3/dist-packages/ansible ansible collection location = /home/workstation/.ansible/collections:/usr/shar e/ansible/collections executable location = /usr/bin/ansible python version = 3.12.3 (main, Jun 19 2026, 12:46:00) [GCC 13.3.0] (/usr/bin/p ython3) jinja version = 3.1.2 libyaml = True workstation@workstation:~\$</pre>														
SSH connectivity														

```
workstation@workstation:~$ ssh santiago@centos
Activate the web console with: systemctl enable --now cockpit.socket

Last login: Thu Aug 13 08:16:57 2026 from 192.168.56.113
[santiago@centos ~]$
logout
Connection to centos closed.
```

```
workstation@workstation:~$ ssh server1@server1
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-134-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Thu Jul 16 09:12:59 2026 from 192.168.56.113
```

```
workstation@workstation:~$ ssh santiago@server2
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.19.0-32-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

13 updates can be applied immediately.
4 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

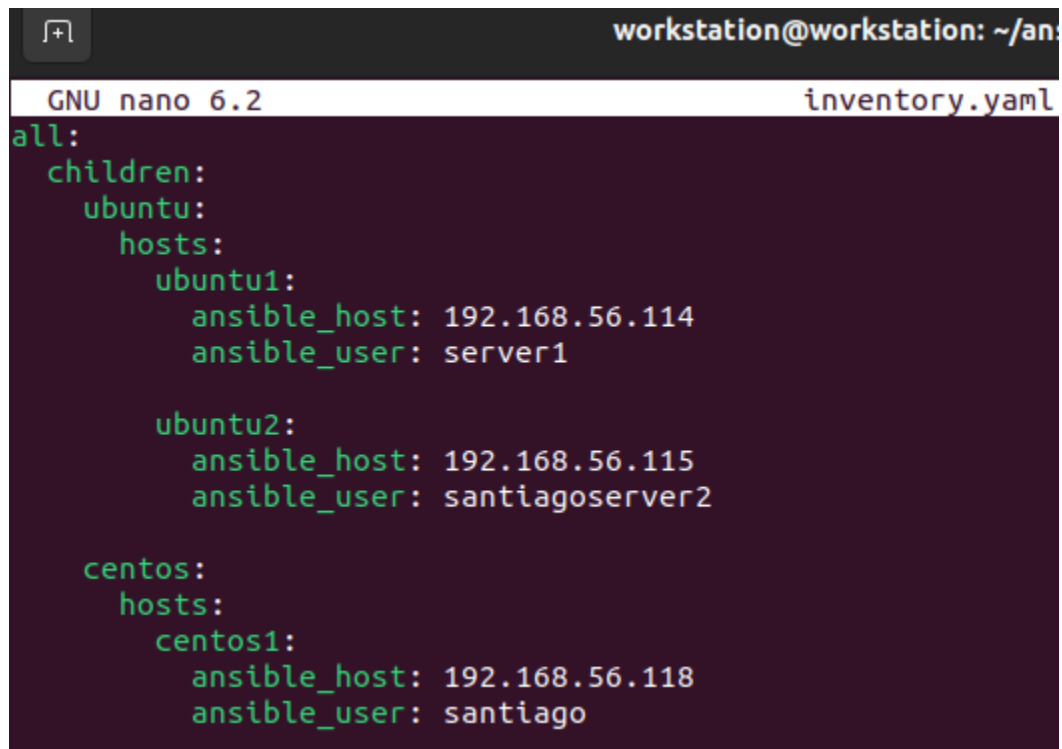
The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Last login: Thu Jul 16 09:13:18 2026 from 192.168.56.113
```

Part B: Building an Inventory File

Inventories provide Ansible with information about managed nodes and their network locations.

Demonstrate the following:

- inventory.yaml

A screenshot of a terminal window with a dark background. The title bar shows 'workstation@workstation: ~/an'. The terminal shows the GNU nano 6.2 editor editing 'inventory.yaml'. The content of the file is as follows:

```
all:
  children:
    ubuntu:
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.114
          ansible_user: server1
        ubuntu2:
          ansible_host: 192.168.56.115
          ansible_user: santiagoserver2
    centos:
      hosts:
        centos1:
          ansible_host: 192.168.56.118
          ansible_user: santiago
```

- Successful inventory verification

```
workstation@workstation:~/ansible-lab$ ansible-inventory -i inventory.yaml --list
{
  "_meta": {
    "hostvars": {
      "centos1": {
        "ansible_host": "192.168.56.118",
        "ansible_user": "santiago"
      },
      "ubuntu1": {
        "ansible_host": "192.168.56.114",
        "ansible_user": "server1"
      },
      "ubuntu2": {
        "ansible_host": "192.168.56.115",
        "ansible_user": "santiagoserver2"
      }
    }
  },
  "all": {
    "children": [
      "ungrouped",
      "ubuntu",
      "centos"
    ]
  },
  "centos": {
    "hosts": [
      "centos1"
    ]
  },
  "ubuntu": {
    "hosts": [
      "ubuntu1",
      "ubuntu2"
    ]
  }
}
```

workstation@workstation:~/ansible-lab\$

- Successful ping results

```
workstation@workstation:~/ansible-lab$ ansible all -m ping -i inventory.yaml
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
centos1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
workstation@workstation:~/ansible-lab$
```

Part C: Creating a Metagroup

Metagroups allow multiple groups to be combined into a single logical group

Demonstrate successful execution of the metagroup.

```
GNU nano 6.2                                inventory.yaml *
all:
  children:
    ubuntu:
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.114
          ansible_user: server1

        ubuntu2:
          ansible_host: 192.168.56.115
          ansible_user: santiagoserver2

    centos:
      hosts:
        centos1:
          ansible_host: 192.168.56.118
          ansible_user: santiago

    linuxservers:
      children:
        ubuntu:
        centos:
```

```
workstation@workstation:~/ansible-lab$ ansible linuxservers -
hosts (3):
  ubuntu1
  ubuntu2
  centos1
workstation@workstation:~/ansible-lab$
```

Part D: Creating Variables

Variables allow information such as operating systems and usernames to be assigned to hosts and groups

```
all:
  children:
    ubuntu:
      vars:
        ansible_user: santiagoserver2
      hosts:
        ubuntu1:
          ansible_host: 192.168.56.114
          ansible_user: server1
          os_type: Ubuntu

        ubuntu2:
          ansible_host: 192.168.56.115
          #ansible_user: santiagoserver2
          os_type: Ubuntu

    centos:
      hosts:
        centos1:
          ansible_host: 192.168.56.118
          os_type: CentOS

  linuxservers:
    children:
      ubuntu:
      centos:
      vars:
        ansible_user: santiago
```

```
workstation@workstation:~/ansible-lab$ ansible all -m ping -i inventory.yaml
ubuntu2 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
ubuntu1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
centos1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python3"
  },
  "changed": false,
  "ping": "pong"
}
workstation@workstation:~/ansible-lab$
```

```
workstation@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml testvars.yaml

PLAY [Display Variables] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]
ok: [ubuntu1]
ok: [centos1]

TASK [Show Operating System] *****
ok: [ubuntu1] => {
  "msg": "Host ubuntu1 is running Ubuntu"
}
ok: [ubuntu2] => {
  "msg": "Host ubuntu2 is running Ubuntu"
}
ok: [centos1] => {
  "msg": "Host centos1 is running CentOS"
}

PLAY RECAP *****
centos1 : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0    ignored=0
ubuntu1 : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0    ignored=0
ubuntu2 : ok=2    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0    ignored=0
workstation@workstation:~/ansible-lab$
```

Part E: Creating Your First Playbook

Playbooks are YAML files used to automate tasks on managed nodes.

```
workstation@workstation:~/ansible-lab$ nano playbook.yaml
workstation@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml playbook.yaml

PLAY [First Ansible Playbook] *****

TASK [Gathering Facts] *****
ok: [ubuntu2]
ok: [ubuntu1]
ok: [centos1]

TASK [Ping all managed nodes] *****
ok: [ubuntu1]
ok: [ubuntu2]
ok: [centos1]

TASK [Display Hello World] *****
ok: [ubuntu1] => {
  "msg": "Hello World from ubuntu1"
}
ok: [ubuntu2] => {
  "msg": "Hello World from ubuntu2"
}
ok: [centos1] => {
  "msg": "Hello World from centos1"
}

TASK [Display Operating System] *****
ok: [ubuntu1] => {
  "msg": "Operating System: Ubuntu"
}
ok: [ubuntu2] => {
  "msg": "Operating System: Ubuntu"
}
ok: [centos1] => {
  "msg": "Operating System: CentOS"
}

PLAY RECAP *****
centos1      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0  ignored=0
ubuntu1      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0  ignored=0
ubuntu2      : ok=4    changed=0    unreachable=0    failed=0    skipped=0    rescue=0
ed=0  ignored=0
workstation@workstation:~/ansible-lab$
```

Guide Questions:

1. What is the role of the Control Node in Ansible?

The control node manages and handles all of the managed nodes in an ansible environment. This manages the inventory files and the playbooks for automated management.

2. Why is an inventory file required?

An inventory file is required to define the nodes that will be managed by the control nodes. It contains the relevant information like os_type, username, and most importantly, the IP address.

3. What is the purpose of a metagroup?

Metagroups are groups of hosts in an inventory. Tasks can be assigned to separate groups instead of individually assigning them in the playbooks.

4. How do variables simplify Ansible administration?

Defined variables help in automating connections for requirements like user names. A global variable that is applicable to all nodes will remove the need for manual assignment.

5. What is the difference between an ad hoc command and a playbook?

An ad-hoc command is a one-time use command in the CLI for executing tasks on managed nodes. Meanwhile, playbooks are files containing defined tasks that can be automated and reused.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

New server group:

```
servers:
  hosts:
    ubuntu1:
    centos1:
```

Challenge.yaml file:

```
! inventory.yaml ! testvars.yaml ! playbook.yaml ! challenge.yaml X
! challenge.yaml
1 ---
2 - name: Challenge YAML
3   hosts: servers
4   tasks:
5     - name: Display Hostname
6       debug:
7         msg: "Welcome to Ansible Laboratory from {{ inventory_hostname }}"
8
```

Running challenge.yaml:

```
workstation@workstation:~/ansible-lab$ ansible-playbook -i inventory.yaml challenge.yaml

PLAY [Challenge YAML] *****

TASK [Gathering Facts] *****
ok: [ubuntu1]
ok: [centos1]

TASK [Display Hostname] *****
ok: [ubuntu1] => {
  "msg": "Welcome to Ansible Laboratory from ubuntu1"
}
ok: [centos1] => {
  "msg": "Welcome to Ansible Laboratory from centos1"
}

PLAY RECAP *****
centos1 : ok=2  changed=0  unreachable=0  failed=0  skipped=0  rescu
ed=0  ignored=0
ubuntu1 : ok=2  changed=0  unreachable=0  failed=0  skipped=0  rescu
ed=0  ignored=0

workstation@workstation:~/ansible-lab$
```

4. CONCLUSION/ANALYSIS/LEARNINGS

In conclusion, this exam shows the installation and use of Ansible for server management. It simplifies system administration because it can automate tasks on groups of nodes. It is useful for streamlining management because an automated task can be assigned to specific groups of nodes. Moreover, tasks can be reused, remodified, and containerized through playbook files.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.



COMPUTER
ENGINEERING

QUEZON CITY

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.